

Harness Engineering for LLM Agents: A Unified Reference Model, Comparative Analysis of Self-Evolving Harnesses, and a Reporting Standard

First A. Author¹, Second B. Author¹, Third C. Author²

¹Department of Computer Science, University Name, City, Country

²Department of Software Engineering, University Name, City, Country

Email: {first.author, second.author}@univ.edu, third.author@univ.edu

Abstract—An LLM agent is not a model alone: it is a model coupled to a harness, the model-external runtime that constructs context, exposes tools, mediates actions, persists state, verifies results, and recovers from failure. With weights held fixed, changing only the harness moves task success by double-digit margins—for example, GPT-5 solves 35.2% of Terminal-Bench 2.1 inside Terminus 2 but 49.6% inside Codex CLI. Since the term “harness engineering” was named in early 2026, the field has produced at least four mutually incompatible taxonomies, four automated harness-evolution systems, and two large-scale empirical studies, yet no shared vocabulary or comparability protocol. This paper addresses that fragmentation. Our contributions are fourfold. First, we formalize the Unified Harness Reference Model (UHRM), a canonical decomposition defined by sound projections and injectivity, and derive a 12-dimensional model that reconciles four independent taxonomies; the crosswalk exposes which concerns each taxonomy omits. Second, we define a ten-axis comparative protocol for self-evolving harness systems—adaptation locus, trigger, evidence, label access, selection rule, weight updates, persistence, transfer, failure modes, and safety reporting—and apply it to Agentic Harness Engineering, Self-Harness, TTHE, and HarnessDev. Third, we re-normalize every harness gain these systems report into a common effect scale, and show that the apparent ordering is an artifact of evaluation regime: test-time adaptation is transductive, creation studies use visible feedback, and slice-conditioned benchmarks inflate relative gains. Fourth, we propose the Harness Evolution Reporting Standard (HERS), extending the Harness Condition Sheet with evolution-specific fields. Across all four systems, no governance or safety metric is reported for self-modified harnesses—a structural gap that we develop into a falsifiable research agenda.

Index Terms—harness engineering, LLM agents, coding agents, agent architecture, self-evolving agents, evaluation methodology, reproducibility, taxonomy

I. Introduction

For most of the short history of large language model (LLM) agents, capability was attributed to the model. That attribution no longer holds for long-horizon work. An agent is a triple

$$\mathcal{A} = \langle M, H, E \rangle, \quad (1)$$

where M is the base model, E the execution environment, and H the harness: the model-external runtime that determines which context is visible, which actions are available, where execution occurs, what state persists, how verification is performed, how failures are recovered, how work is delegated, and where governance is enforced [7].

The empirical case for treating H as an object of study is now strong. HarnessDev reports that with identical weights, GPT-5 solves 35.2% of Terminal-Bench 2.1 inside one harness (Terminus 2) and 49.6% inside another (Codex CLI) [6]—a harness-induced swing of 14.4 percentage points, larger than the separation between many adjacent model generations. Self-Harness shows that editing only the harness raises pass rates on nine model–benchmark combinations, with a largest absolute gain of 40.6 points [4]. Agentic Harness Engineering (AHE) lifts Terminal-Bench 2 pass@1 from 69.7% to 77.0%, surpassing a human-designed harness [3]. Test-Time Harness Evolution (TTHE) improves a fixed ReAct [9] baseline across five domains without any label access [5].

This activity has produced a vocabulary problem. The discipline was named in vendor engineering reports in early 2026 [19], [20] and given conceptual framings shortly afterwards, including a governance-oriented one [8] and a concurrent source-level taxonomy of thirteen open-source scaffolds [18]. At least four independent decompositions of the harness now coexist:

- the seven canonical subsystems of the Anatomy study [1] (agent loop, LLM integration, tools, memory/context, safety, orchestration, extensibility);
- the ETCLOVG seven-layer taxonomy of the Agent Harness Engineering survey [2] (Execution, Tooling, Context, Lifecycle/Orchestration, Observability, Verification, Governance);
- the eight-function Agent Harness Control Plane of [7] (context ingress, action mediation, execution substrate, state persistence, verification, recovery, delegation, governance);
- the six-component view reported by Meng et al. and

adopted in [2] (execution loop, tool registry, context manager, state store, lifecycle hooks, evaluation interface).

These decompositions are neither refinements nor coarsenings of one another. They disagree about whether observability and governance are first-class, whether recovery is distinct from verification, whether the execution substrate is a subsystem at all. Consequently, results reported under one decomposition are not directly interpretable under another, and the four automated evolution systems—AHE, Self-Harness, TTHE, and HarnessDev—each report gains under a different evaluation regime, a different notion of what is editable, and a different treatment of labels.

A. Contributions

This paper is a systematization and comparative analysis. It reports no new benchmark run; every quantitative claim is attributed to the source that published it, and every derived statistic is explicitly marked as our recomputation. We make four contributions.

- 1) A unified reference model (Section III). We formalize what it means for taxonomies of the harness to be compatible, via sound projections and an injectivity requirement, and derive the coarsest canonical model satisfying them. The resulting 12-dimensional UHRM reconciles the four taxonomies above and yields a crosswalk (Table II) that exposes, per taxonomy, which concerns it omits. Model/provider integration and extensibility are named by only one taxonomy each; recovery is first-class in only one; observability in only one.

- 2) A comparative protocol for self-evolving harnesses (Section IV). We define ten axes along which harness-adaptation systems must be described, and apply them to the four systems. The protocol makes explicit a property shared by all four—base weights are frozen—and a property on which they diverge completely: label access.

- 3) A normalized synthesis of reported gains (Section V). We re-express every reported harness gain on a common ceiling-normalized scale γ , and show that the resulting ordering is governed by evaluation regime rather than by method quality. We identify four confounds—transductive evaluation, visible feedback, slice conditioning, and ceiling effects—that the current literature does not control.

- 4) A reporting standard (Section VI). We extend the Harness Condition Sheet of [7] with evolution-specific fields to form the Harness Evolution Reporting Standard (HERS), and we note a uniform omission: none of the four self-evolution systems reports a safety or governance metric for its self-modified harness (Section VII).

II. Background: From Prompt to Harness Engineering

A. What a Harness Is, and Is Not

We adopt the definition of [7]: the harness is the model-external layer that determines which context is visible,

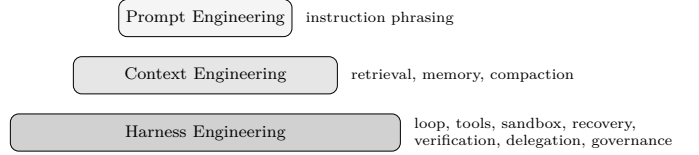


Fig. 1. The widening engineering surface. Harness engineering subsumes prompt and context engineering while adding runtime concerns—execution substrate, recovery, delegation, and governance—that neither earlier stage can express.

which actions are available, where execution occurs, what state persists, how verification is performed, how failures are recovered, how work is delegated, and where governance is enforced. Two boundary cases matter for reading the literature. First, a harness is not an evaluation harness: a SWE-bench harness wraps an agent to run it against tasks, whereas an agent harness wraps a model to make it act—the same word with the direction of wrapping reversed [1]. Second, a harness is not an orchestrator or meta-harness: an orchestration layer such as Omnigent coordinates several harnesses from above but implements no editing loop of its own [1].

Fig. 1 situates harness engineering in the widening sequence from prompt engineering, to context engineering, to the surrounding execution system. The sequence is not a hierarchy of importance but an expanding engineering surface: each stage subsumes the artifacts of the previous one while adding new failure modes that cannot be addressed at the earlier stage.

B. The Four Taxonomies

Table I lists the four decompositions and their dimensions. The Anatomy study derives its seven subsystems from source-level analysis of eleven production coding harnesses plus a meta-harness, across roughly four million lines of Python, TypeScript, and Rust [1]. The ETCLOVG taxonomy is the product of a survey that codes more than 170 open-source projects against seven layers [2]. The Control Plane decomposition is a conceptual framework validated against seven systems and four evidence classes [7]. The six-component view is a component-level description of an individual harness [2].

The taxonomies were derived by different methods (source audit, ecosystem coding, conceptual formalization, component description), for different purposes (describing artifacts, mapping an ecosystem, analyzing control, specifying a single system), and at different granularities. Compatibility is therefore not automatic, which is precisely the problem Section III addresses.

C. Self-Evolving Harness Systems

Four systems automate harness improvement while holding base weights fixed. They are not the first mechanisms to adapt an LLM system without touching weights: verbal self-reflection revises a single response or stores textual experience for later attempts [16], and declarative

TABLE I
The four coexisting decompositions of the agent harness.

Source	Method	Scope	Dimensions
Anatomy et al. [1]	Source-code audit of 12 systems (≈4M LOC)	Artifact anatomy	Agent loop; LLM integration; tools & actions; memory & context; safety & permissions; orchestration; extensibility
ETCLOVG [2]	Coding protocol over 170+ projects	Ecosystem mapping	Execution env.; Tool interface; Context mgmt.; Lifecycle/Orchestration; Observability; Verification; Governance
Control Plane [7]	Conceptual formalization, 7 systems, 4 evidence classes	Control analysis	Context ingress; action mediation; execution substrate; state persistence; verification/review; recovery/debugging; delegation/coordination; governance/audit
Component view [2]	Component description of one harness	Single system	Execution loop; tool registry; context manager; state store; lifecycle hooks; evaluation interface

pipeline compilation searches over prompts and control flow [17]. Both, however, treat the adaptation target as a response or a pipeline rather than as the runtime that mediates the agent’s entire interaction with its environment. AHE [3] exposes seven editable harness component types as files and drives a closed evolution loop through three “observability pillars” (component, experience, and decision observability), where each edit carries a self-declared prediction that is later verified against the next round’s task-level outcomes. Self-Harness [4] runs a three-stage loop—Weakness Mining over clustered execution traces, Harness Proposal of minimal edits tied to identified failures, and Proposal Validation by regression testing. TTHE [5] moves adaptation into evaluation itself, maintaining a population of candidate harnesses, refining them with an agentic proposer over unlabeled execution traces, and committing one via a judge that reads execution-derived proxy signals. HarnessDev [6] is a benchmark rather than a method: it evaluates a model’s ability to Create a runnable harness from a weak seed and then Evolve it from downstream execution feedback.

III. A Unified Harness Reference Model

A. Formalization

Let Ω be the set of model-external, editable components that jointly determine agent behaviour—the harness edit surface. A taxonomy $\tau = \{d_1, \dots, d_k\}$ is a partition of Ω induced by an analyst’s decompositional choices. Two taxonomies τ, τ' are generally incompatible because their blocks were chosen for different purposes.

To compare them we require a common model $U = \{u_1, \dots, u_m\}$ together with a projection $\pi_i : U \rightarrow \tau_i \cup \{\emptyset\}$ for each source taxonomy τ_i , mapping each canonical dimension to at most one source dimension (the one that contains it).

Soundness. For every $d \in \tau_i$, $\bigcup \{u \in U : \pi_i(u) = d\} = d$.
Equivalently, no canonical dimension straddles

two blocks of any source taxonomy: U is a genuine common refinement in the semantic sense, not a re-labeling.

Injectivity. The map $u \mapsto (\pi_1(u), \dots, \pi_n(u))$ is injective. Two canonical dimensions may coexist only if at least one source taxonomy distinguishes them; dimensions that all sources conflate must be merged.

Coarseness. Among all U admitting sound, injective projections, U has minimum cardinality m .

The three conditions together determine U up to renaming: merging any two dimensions is forbidden by injectivity unless their projection vectors coincide, and retaining any two dimensions with coinciding vectors is forbidden by coarseness. The model is thus the coarsest common refinement that no source taxonomy can object to. Because coarseness is stated as minimal cardinality subject to soundness and injectivity, the construction is well-defined for $n \geq 2$; for $n = 1$ the model trivially reproduces that taxonomy.

B. The Twelve-Dimensional Model

Applying the construction to the four taxonomies of Table I yields the twelve canonical dimensions of Table II. Each row is a canonical dimension; each remaining column gives the source dimension to which it projects (– denotes that the source has no correspondent, i.e. the projection is $\emptyset$).

Three consequences follow immediately from the crosswalk, and each is independent of any particular experiment.

(i) The component view is the least complete. The six-component description adopted in [2] has no correspondent for D1, D3, D9, D10, D11, or D12—that is, for the execution substrate, model integration, recovery, observability, governance, and extensibility. A system described only in component terms cannot express the security surface

TABLE II

The Unified Harness Reference Model (UHRM) and its crosswalk to four coexisting taxonomies. A dash denotes that the source taxonomy has no corresponding dimension. Recomputed by us from the definitions in [1], [2], [7].

#	UHRM dimension	Anatomy [1]	ETCLOVG [2]	Control Plane [7]
D1	Execution substrate & sandboxing	Safety & perms. (part.)	Execution env. (E)	Execution substrate (X)
D2	Agent loop & control flow	Agent loop (part.)	Lifecycle/Orch. (part.)	— (data plane)
D3	Model & provider integration	LLM integration	—	—
D4	Tool & action interface	Tools & actions	Tool interface (T)	Action mediation (A)
D5	Context management	Memory & context (part.)	Context (C)	Context ingress (C)
D6	Memory & state persistence	Memory & context (part.)	Context/Lifecycle (part.)	State persistence (S)
D7	Orchestration & delegation	Orchestration	Lifecycle/Orch. (part.)	Delegation (D)
D8	Verification & evaluation	Agent loop (part.)	Verification (V)	Verification (V)
D9	Recovery & debugging	Agent loop (part.)	Observability (part.)	Recovery (R)
D10	Observability & operations	Session substrate	Observability (O)	Recovery (part.)
D11	Safety, permissions & governance	Safety & permissions	Governance (G)	Governance (G)
D12	Extensibility & extension surfaces	Extensibility	—	—

(D1, D11), the recovery behaviour (D9) that [7] argues is a first-class outcome, or the extension surface (D12) that [1] finds to be the locus of the field’s platform turn.

(ii) Four canonical dimensions are named by exactly one taxonomy each. Model/provider integration (D3) and extensibility (D12) appear only in [1]; observability (D10) is first-class only in [2]; recovery (D9) is first-class only in [7]. A field that adopts any single taxonomy therefore inherits that taxonomy’s blind spot. Cross-vendor portability (D3) is a concrete instance: it is invisible to ETCLOVG and to the Control Plane, yet [1] documents provider-specific cache boundaries, extended-thinking budgets, and a server-delivered model catalog as first-class engineering.

(iii) Observability and governance are separable, and separating them changes what is measured. ETCLOVG is the only taxonomy that treats Observability (D10) and Verification (D8) as distinct layers; the Control Plane folds debugging partly into recovery (D9) and has no observability function; the Anatomy study folds verification and recovery into the agent loop. The distinction is not cosmetic. Under the ETCLOVG separation, a harness can be verified but unobservable—it produces correct outcomes whose provenance cannot be inspected—whereas under the Anatomy decomposition that state is not expressible. This matters for the systems of Section IV, whose selection rules depend on evidence that must first be observed.

IV. Self-Evolving Harnesses: A Comparative Protocol

Existing comparisons of harness-adaptation systems are made difficult by the absence of shared descriptors. We propose a ten-axis protocol: any system that modifies a harness must state (A1) the adaptation locus, i.e. which UHRM dimensions are editable; (A2) the trigger, i.e. when adaptation occurs relative to deployment; (A3) the evidence source; (A4) label access; (A5) the selection rule that accepts or reverts a candidate; (A6) whether weights are updated; (A7) the persistence of the adapted artifact; (A8) the transfer protocol; (A9) reported failure

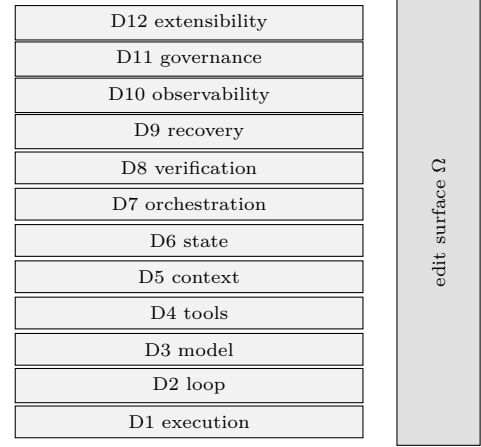


Fig. 2. UHRM as a canonical partitioning of the harness edit surface. The model is the coarsest partition admitting sound, injective projections from all four source taxonomies.

modes; and (A10) reported safety and governance metrics. Table III applies the protocol to the four systems.

A. Two Convergences and One Hard Divergence

Convergence 1: weights are frozen in all four systems. This is what distinguishes harness engineering from fine-tuning or test-time training. It is also what makes the reported gains attributable to H rather than to M , provided the harness condition is fully disclosed—the caveat that motivates Section VI.

Convergence 2: all four select on execution-derived evidence rather than on human judgement. AHE verifies self-declared predictions against next-round outcomes, Self-Harness regression-tests, TTHE judges on proxy signals, and HarnessDev requires formal evaluation completion. Selection is therefore only as reliable as the evidence, which is the binding constraint the systems themselves identify.

Divergence: label access partitions the field. The four systems occupy three incompatible regimes. AHE and Self-Harness assume access to task-level labels during adapta-

TABLE III

Ten-axis comparison of self-evolving harness systems. All values are taken from the cited papers; “n/r” denotes not reported.

Axis	AHE [3]	Self-Harness [4]	TTHE [5]	HarnessDev [6]
A1 Locus	7 editable component types exposed as files (prompt, tools, middleware, memory, ...)	Declared harness interfaces (prompt, tools, run-time, verification)	Executable harness program: construction, tool grounding, contract checks, recovery branches	Entire runnable harness codebase (multi-file)
A2 Trigger	Iterative offline rounds (10 iterations)	Iterative offline loop (3 stages)	Test time, per unlabeled batch	Creation then Evolution; both pre-deployment
A3 Evidence	Layered, drill-down trajectory corpus distilled from millions of tokens	Clustered execution traces (weakness mining)	Unlabeled execution traces + execution-derived proxy signals	Downstream execution feedback (SWE-Pro-100, Terminal-Bench-89)
A4 Labels	Task-level outcomes, consulted after edit	Benchmark pass labels on held-in split	Label-free; gold used only for measurement	Visible feedback set; 630-instance held-out split withheld
A5 Selection	Change manifest: keep if self-declared prediction verified next round; revert at file granularity	Accept only edits that pass regression testing	Agentic judge commits one candidate from proxy signals	Official version must complete both formal evaluations
A6 Weights	Frozen	Frozen	Frozen	Frozen
A7 Persistence	Persistent files	Persistent harness	Selected program governs subsequent inputs	Frozen, versioned, runnable harness
A8 Transfer	SWE-bench-Verified; 3 alternate model families (+5.1 to +10.1 pp)	Held-in and held-out splits, 3 benchmarks $\times$ 3 models	Transductive (same batch); cross-model check on BIRD	Held-out-630; fixed-executor (Gemini) transfer
A9 Failure modes	Components interact non-additively; self-attribution blind to regressions	n/r	Non-monotonic in search budget; judge selection regret; proxy unreliability	Unstable gains; partial transfer; executor-specific brittleness
A10 Safety	n/r	n/r	n/r	n/r

tion. HarnessDev assumes a visible feedback set drawn from the same distribution as the held-out split. TTHE assumes no labels and adapts on the test batch itself. A gain reported under one regime carries no implication for another: a system that optimizes against labels can in principle be strictly more effective than a label-free system while telling us nothing about label-free adaptation, and vice versa. We return to the consequence in Section V.

B. The Adaptation-Time Axis

Fig. 3 places the four systems on the two axes that most strongly determine comparability: when adaptation occurs and what supervision it consumes. HarnessDev’s Creation stage is the only point that asks a model to build a harness from a weak seed; TTHE is the only point that adapts without labels. The lower-right region—test-time, label-free, held-out evaluation—is empty, and that emptiness is itself a finding: no system demonstrates label-free test-time adaptation measured on data it did not adapt to.

V. Synthesis of Reported Gains

A. A Common Effect Scale

Absolute point gains are not comparable across benchmarks with different headroom. We therefore re-express each reported gain on a ceiling-normalized scale. For an initial harness score S_0 and a final score S_1 on the same benchmark, model, and metric, we use

$$\gamma = \frac{S_1 - S_0}{100 - S_0}, \quad \rho = \frac{S_1 - S_0}{S_0}, \quad (2)$$

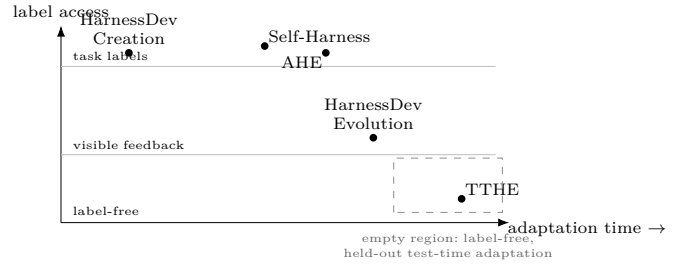


Fig. 3. The adaptation-time/supervision plane. The four systems occupy three mutually non-comparable regimes. The dashed region—label-free test-time adaptation evaluated on unseen data—is unoccupied.

where γ is the fraction of the remaining headroom to a perfect score that the harness recovers, and ρ is the relative gain. Both are computed by us from published numbers; neither is reported by the source systems. Table IV lists every reported harness gain we could normalize across the benchmarks used by the corpus: BIRD [14], Live-CodeBench [15], SWE-bench Verified [10], DS-1000 [13], AppWorld [12], and Terminal-Bench [11].

B. The Apparent Ordering Is an Evaluation Artifact

Read naively, Table IV suggests that TTHE dominates: it reports the largest absolute gain and four of the highest ρ values, without labels. Normalizing changes the picture. The mean ceiling-normalized gain is $\bar{\gamma} = 0.268$ for Self-Harness (nine cells) and $\bar{\gamma} = 0.249$ for TTHE (five cells,

TABLE IV

Reported harness gains re-expressed on a common scale. S_0 and S_1 are the initial and final harness scores exactly as published; Δ , ρ , and γ are recomputed by us. Metrics differ across rows (pass@1, official-verifier pass rate, graded score), so rows are comparable only within a benchmark.

System	Benchmark	Model	$S_0 \rightarrow S_1$	Δ	ρ	γ
AHE [3]	Terminal-Bench 2	(frozen base)	69.7 $\rightarrow$ 77.0	+7.3	0.105	0.241
Self-Harness [4]	Terminal-Bench-2.0	MiniMax M2.5	42.2 $\rightarrow$ 53.9	+11.7	0.277	0.202
Self-Harness [4]	Terminal-Bench-2.0	Qwen3.5-35B-A3B	18.0 $\rightarrow$ 36.7	+18.7	1.039	0.228
Self-Harness [4]	Terminal-Bench-2.0	GLM-5	46.1 $\rightarrow$ 57.0	+10.9	0.236	0.202
Self-Harness [4]	SWE-bench Verified	MiniMax M2.5	46.0 $\rightarrow$ 52.5	+6.5	0.141	0.120
Self-Harness [4]	SWE-bench Verified	Qwen3.5-35B-A3B	19.5 $\rightarrow$ 41.5	+22.0	1.128	0.273
Self-Harness [4]	SWE-bench Verified	GLM-5	52.0 $\rightarrow$ 55.5	+3.5	0.067	0.073
Self-Harness [4]	AppWorld	MiniMax M2.5	48.6 $\rightarrow$ 58.9	+10.3	0.212	0.200
Self-Harness [4]	AppWorld	Qwen3.5-35B-A3B	22.5 $\rightarrow$ 52.2	+29.7	1.320	0.383
Self-Harness [4]	AppWorld	GLM-5	44.4 $\rightarrow$ 85.0	+40.6	0.914	0.730
TTHE [5]	BIRD (hard slice)	DeepSeek V4 Flash	12.0 $\rightarrow$ 50.0	+38.0	3.167	0.432
TTHE [5]	LiveCodeBench	DeepSeek V4 Flash	30.0 $\rightarrow$ 38.3	+8.3	0.277	0.119
TTHE [5]	SWE-bench Verified	DeepSeek V4 Flash	20.0 $\rightarrow$ 35.0	+15.0	0.750	0.188
TTHE [5]	DS-1000	DeepSeek V4 Flash	38.0 $\rightarrow$ 44.0	+6.0	0.158	0.097
TTHE [5]	claw-eval (graded)	DeepSeek V4 Flash	48.9 $\rightarrow$ 69.8	+20.9	0.427	0.409
HarnessDev [6]	5-benchmark mean (Creation)	Opus 4.8	0.0 $\rightarrow$ 67.8	+67.8	—	0.678

including the different-metric claw-eval row; excluding it, 0.209), while AHE contributes a single datum at $\gamma = 0.241$. These means lie well within one standard deviation of one another—the standard deviation of γ within Self-Harness alone is 0.183—and the spread within Self-Harness (0.073 to 0.730) exceeds the spread between the three systems (0.209 to 0.268). The apparent ordering is therefore not supported once headroom is accounted for.

Four confounds explain most of the residual differences.

Confound 1: transductive evaluation. TTHE scores each batch with the harness committed for that batch, so the adapted artifact is measured on the data it adapted to [5]. The authors state this explicitly and its magnitude should not be compared to the held-out figures of other systems. Rows for AHE (cross-family transfer), Self-Harness (held-out splits), and HarnessDev (held-out-630) are of a different kind.

Confound 2: slice conditioning. TTHE’s BIRD slice is constructed from 50 questions the baseline repeatedly failed [5]. Its 12.0% start is therefore low by construction, and its +38.0 gain measures recovery on known-hard queries rather than a general improvement. This row alone contributes the highest ρ in the table (3.167) and would be misleading if quoted without the conditioning.

Confound 3: visible feedback versus withheld data. HarnessDev’s Evolution feedback set is shown to the creator, and the authors report that gains shrink on held-out tasks: the largest held-out improvement is +4.44 points (Opus 4.8), and under a fixed Gemini executor only Opus improves on held-out data while other lineages regress [6]. The Creation row in Table IV is a different regime again—it measures construction from a zero-scoring seed, not evolution.

Confound 4: ceiling and floor effects. Normalization by $(100 - S_0)$ is a convenience, not a model of achievable

headroom. Where a human-engineered reference exists it is a better ceiling. HarnessDev reports human references of 80.0 on SWE-Pro and 88.8 on Terminal-Bench 2.1, with the best created harness at 67.8 on a five-benchmark mean against a human mean of 86.2 [6]; against the human ceiling, the gap is 18.4 points. A ceiling estimated from human performance would change γ for every row in Table IV, and no source system justifies its chosen denominator.

C. Harness Sensitivity and Non-Additivity

Two findings in the corpus constrain how much of a gain can ever be attributed to the harness. First, harness sensitivity can rival model replacement: the GPT-5 datum (35.2% vs 49.6% on Terminal-Bench 2.1 across two harnesses, weights fixed [6]) exceeds the +7.3 point gain AHE obtains from ten evolution iterations. Harness choice and harness evolution are thus different levers, and the former is currently the larger one.

Second, harness components interact non-additively. AHE’s ablation localizes its gain to tools, middleware, and long-term memory, while the system prompt alone regresses performance [3]; the authors conclude that stacking individually effective edits caps the aggregate gain. This is the evolution-loop analogue of the coupling problem identified at ecosystem scale in [2]: local optimization of a coupled control system is fragile, and a component that looks beneficial in isolation can degrade the whole rollout.

D. Efficiency Is Reported but Not Correlated with Quality

Harness adaptation has a cost, and the reported evidence does not support “more spend, better harness.” AHE’s frozen harness uses 12% fewer tokens than its seed on SWE-bench-Verified while achieving the highest aggregate success [3]. HarnessDev reports roughly nineteen-fold variation in MLE-bench token use across creators that does not reliably predict score; Gemini adds the fewest

lines (1,006) yet obtains the best Terminal-Bench score (68.8), while the 18 code artifacts together add 17,111 net lines [6]. Edit size and token spend are therefore poor proxies for harness quality, which strengthens the case for the reportable-budget field in HERS.

VI. A Reporting Standard for Harness Evolution

The Harness Condition Sheet of [7] requires that a benchmark run or deployment study record context artifacts, tool and protocol exposure, permission and network policy, execution substrate, persistence features, verification stack, recovery mechanisms, delegation topology, human approval rules, and cost or time budgets. This is necessary but insufficient for evolution studies, which must additionally disclose what was edited, on what evidence, and under what supervision. Table V specifies the Harness Evolution Reporting Standard (HERS): the original condition fields, plus twelve evolution-specific fields drawn from the axes of Section IV.

VII. Open Problems and Research Agenda

1) Governance of self-modifying harnesses. The most uniform omission in the corpus is also the most consequential. Table III, axis A10, is “not reported” for every system: none of AHE, Self-Harness, TTHE, or HarnessDev reports an unsafe-action rate, a permission-boundary violation, or a human-intervention burden for its self-modified harness. This is not a criticism of individual papers—none set out to measure governance—but it is a structural gap. A system that edits the layer through which an agent obtains authority is exactly the system for which the authority delta must be measured. The Control Plane framework already formalizes the required quantities (unsafe action rate, human intervention burden) [7]; what is missing is their application to evolved harnesses. A minimal first study would report the UHRM dimensions touched by each accepted edit, the permission-relevant surface exposed after adaptation, and the change in unsafe-action rate relative to the seed under a fixed adversary model. The survey’s call for unified adversarial benchmarks for governance stacks [2] supplies the adversary model.

2) Selection under unreliable proxies. TTHE identifies execution-derived proxy reliability as a central challenge and quantifies it: a pool oracle shows roughly fourteen points already generated but left uncommitted, i.e. selection regret rather than generation failure [5]. Reported harness performance is non-monotonic in search budget, with more rounds helping only when more proposers are used. The open question is precise: given a fixed budget and a proxy whose fidelity is unknown, what selection rule maximizes held-out gain? This is a bandit problem with a biased reward signal, and none of the four systems states its proxy’s calibration.

3) Regression foresight. AHE reports that its loop’s self-attribution is reliable for fixes but blind to regressions [3]; HarnessDev reports that evolution gains are unstable and

TABLE V
The Harness Evolution Reporting Standard (HERS). Group A extends the Harness Condition Sheet of [7]; Group B is new and required for any claim of harness improvement.

ID	Field	Requirement
Group A — Harness condition (after [7])		
A1	Context artifacts	Files, memory, skills, retrieval sources
A2	Tool/protocol exposure	Tool inventory; MCP/ACP/A2A [21] surfaces
A3	Permission & network policy	Approval rules; allow/deny lists; egress
A4	Execution substrate	Sandbox type; OS isolation; reset semantics
A5	Persistence features	State store; checkpointing; resume/fork
A6	Verification stack	Verifiers; synchronous vs. offline checks
A7	Recovery mechanisms	Retry policy; rollback; failure taxonomy
A8	Delegation topology	Sub-agents; roles; handoff protocol
A9	Human approval rules	Gating points; intervention budget
A10	Cost/time budget	Token, wall-clock, and step limits
Group B — Evolution disclosure (new)		
B1	Adaptation locus	UHRM dimensions edited (Table II)
B2	Adaptation trigger	Creation / offline rounds / test time
B3	Editable surface size	Files, LOC, parameters exposed
B4	Evidence source	Traces, feedback, proxy signals
B5	Label access	None / held-in / visible feedback
B6	Selection rule	Acceptance and revert semantics
B7	Rollback semantics	Granularity of reversion
B8	Weight-update status	Frozen or updated (with justification)
B9	Artifact persistence	Versioning; reproducibility of the artifact
B10	Transfer protocol	Held-out splits; cross-model; cross-executor
B11	Adaptation budget	Iterations, rollouts, proposer tokens
B12	Safety & governance metrics	Unsafe-action rate; approval burden

transfer only partially, with lineages regressing under a fixed executor [6]. Both point to the same missing capability: predicting, before committing an edit, which previously passing behaviours it will break. Self-Harness’s regression-testing acceptance rule [4] is the only mechanism in the corpus aimed at this, and it is a filter rather than a predictor. A predictor would need per-edit impact attribution, which HERS field B1 is designed to make possible.

4) Comparability by construction. The confounds of Section V are not incidental. No benchmark in the corpus varies the harness while holding the model, data, and evaluation protocol fixed across multiple evolution

methods. Terminal-Bench 2 is used by AHE and Self-Harness under different base models and different harnesses; HarnessDev’s contribution is precisely to make the harness the unit of evaluation, but it evaluates creators, not evolution algorithms head-to-head. A harness-ablation benchmark—fixed model, fixed task distribution, multiple adaptation algorithms, held-out evaluation, mandated HERS reporting—would let the field answer whether any evolution method dominates another.

5) Recovery and state persistence. HarnessDev finds that while 11 of 18 created code harnesses define a state class, only one exposes a state-saving interface and only one implements periodic checkpointing; no checkpoint event appears across 26,679 recorded task trajectories [6]. Recovery is the least-implemented UHRM dimension relative to its importance for long-horizon tasks, and it is precisely the dimension that the Control Plane frames as a first-class outcome alongside one-shot success [7]. Whether recoverability is better predicted by loop design (D2) or by state persistence (D6) is an open empirical question.

6) Termination and saturation. AHE’s ten iterations and the non-monotonic budget curves of TTHe both imply that evolution has a stopping problem: gains are not monotone in effort, and the systems do not state a principled stopping criterion. As base models improve, the harness headroom shrinks for reasons unrelated to harness quality—which motivates the survey’s open question of how to simplify harnesses as models improve [2]. A saturation-aware objective would treat harness complexity as a cost, not only token spend.

VIII. Threats to Validity

Secondary analysis. Every number in Table IV is re-computed from a published figure; we ran no experiment. Where a source reports a distribution (e.g. avg@3 over three independent harnesses in [6]) we used its reported mean, so our γ inherits the source’s variance, which we do not propagate. Heterogeneous metrics. Rows of Table IV span pass@1, official-verifier pass rates, and a graded score in $[0, 1]$; normalization by a 100-point ceiling is a convenience that assumes a comparable scale, and the claw-eval row is explicitly not comparable to the others. Taxonomy construction. The UHRM depends on the semantic alignment encoded in the projections π_i ; a reader who disputes an alignment would obtain a different crosswalk. We mitigate this by stating the soundness, injectivity, and coarseness conditions explicitly so that any disputed cell can be revised without changing the construction. Preprint status. All sources are 2026 preprints or technical reports; several may be revised, and the quantitative claims of Section V track the versions cited. Selection of systems. We compare the four self-evolving systems known to us at the time of writing; the corpus is young and the comparison is not claimed to be exhaustive.

IX. Conclusion

An LLM agent is a model plus a harness, and the harness has become a first-class engineering surface: with weights frozen, harness choice alone moves Terminal-Bench 2.1 success by 14.4 points. This paper argued that the field’s rapid growth has outrun its vocabulary. We formalized compatibility between harness taxonomies via sound, injective projections, and derived the 12-dimensional UHRM, whose crosswalk shows that each of the four leading taxonomies omits concerns the others name—model integration, observability, recovery, extensibility. We defined a ten-axis protocol for harness-adaptation systems and found that all four self-evolving systems freeze weights, all select on execution-derived evidence, and all diverge on the single axis that most determines comparability: label access. Re-normalizing every reported gain onto a common headroom scale dissolved the apparent ranking between systems, and the residual differences were attributable to four confounds—transductive evaluation, slice conditioning, visible feedback, and unspecified ceilings—that we proposed HERS to control. The sharpest finding is a uniform omission: no self-evolution system reports a safety or governance metric for the harness it modified. Measuring the authority delta of a self-modified harness is, in our view, the most urgent open problem in the field.

References

- [1] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, “Harness engineering: Anatomy, architecture, and evolution of coding agents — a source-code study of eleven systems,” arXiv:2609.00006, Jul. 2026.
- [2] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, et al., “Agent harness engineering: A survey,” 2026. [Online]. Available: <https://picrew.github.io/LLM-Harness/>
- [3] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, “Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses,” arXiv:2604.25850, May 2026.
- [4] H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu, “Self-Harness: Harnesses that improve themselves,” arXiv:2606.09498, Aug. 2026.
- [5] J. Nie, Y. Zhang, J. Song, Q. Cai, D. Yu, Y. Guo, X. Tian, and B. Han, “TTHe: Test-time harness evolution,” arXiv:2607.08124, Jul. 2026.
- [6] ByteDance Seed et al., “HarnessDev: Can LLMs create and evolve their own agent harness?” arXiv:2609.01437, Sep. 2026.
- [7] C. Mishra, “Harness-native software engineering: The control plane of coding agents,” Mar. 2026. [Online]. Available: <https://research.chaitanya.science/>
- [8] J. Kim, “Harness engineering: A governance framework for AI-driven software engineering,” Zenodo, Mar. 2026, doi: 10.2139/ssrn.6372119.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. ICLR, 2023.
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in Proc. ICLR, 2024.
- [11] The Terminal-Bench Team, “Terminal-Bench: A benchmark for AI agents in terminal environments,” Laude Institute, 2026. [Online]. Available: <https://www.tbench.ai/>

- [12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in Proc. ACL, 2024.
- [13] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, S. W. Yih, D. Fried, S. Wang, and T. Yu, “DS-1000: A natural and reliable benchmark for data science code generation,” in Proc. ICML, 2023.
- [14] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., “Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs,” in Proc. NeurIPS, 2023.
- [15] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “LiveCodeBench: Holistic and contamination-free evaluation of large language models for code,” in Proc. ICLR, 2025.
- [16] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in Proc. NeurIPS, 2023.
- [17] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, et al., “DSPy: Compiling declarative language model calls into self-improving pipelines,” in Proc. ICLR, 2024.
- [18] B. Rombaut, “Inside the scaffold: A source-code taxonomy of coding agent architectures,” arXiv preprint, 2026.
- [19] OpenAI, “Harness engineering: Leveraging Codex in an agent-first world,” 2026. [Online]. Available: <https://openai.com/index/harness-engineering/>
- [20] Anthropic, “Harness design for long-running application development,” 2026. [Online]. Available: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- [21] Anthropic, “Model Context Protocol,” 2024. [Online]. Available: <https://modelcontextprotocol.io/>